

TravelMemory

MERN Stack Deployment on Amazon EC2

Deployment Documentation with Screenshots

Live application: <https://trendlycart.in>

Source repo: github.com/UnpredictablePrashant/TravelMemory

Submitted by: ____NEV Rajani Kumar____ Date: ____28-Jun-26____

Table of Contents

1. Introduction	3
2. Deployment Architecture	4
3. Backend & Database Configuration	4
4. nginx Reverse Proxy & Security Group	5
5. Frontend & Backend Connection	6
6. Scaling the Application	8
7. Domain Setup with Cloudflare	10
8. Verification Summary	11
9. Challenges & Resolutions	12
10. Deliverables & Submission	12

1. Introduction

The TravelMemory application is a full-stack travel journal built on the MERN stack (MongoDB, Express, React, Node.js). This document details its deployment on Amazon EC2: backend on Node.js, frontend in React, communication between the tiers via an nginx reverse proxy, horizontal scaling using a second instance launched from an AMI, and exposure through a custom domain on Cloudflare. The application is live over HTTPS at <https://trendlycart.in>.

Environment Summary

Component	Value / Choice
Cloud platform	AWS EC2 — Asia Pacific (Mumbai), ap-south-1
Operating system	Amazon Linux 2023
Primary instance	devops-assignment (i-028b7ea52e4aa4f6c), 13.232.19.204
Second instance (from AMI)	new_instance_fromami, 13.127.58.45
Backend	Node.js / Express on port 3001 (systemd service)
Frontend	React production build served by nginx
Database	MongoDB Community Edition installed on the EC2 instance
Domain & registrar	trendlycart.in (Hostinger)
DNS & TLS	Cloudflare (proxied, Flexible SSL)

Note: The brief states port 3000 and a file named `urls.js`. The actual repository runs the backend on port 3001 (`backend/index.js` reads `process.env.PORT`) and the frontend file is `frontend/src/url.js`, which exports `baseUrl` from `REACT_APP_BACKEND_URL`. This deployment follows the real code.

2. Deployment Architecture

The diagram below (created in draw.io) shows the flow from the user through Cloudflare to the EC2 instances. Cloudflare provides DNS resolution and edge HTTPS; nginx serves the React build and reverse-proxies API calls to the Node backend, which connects to MongoDB on the instance.

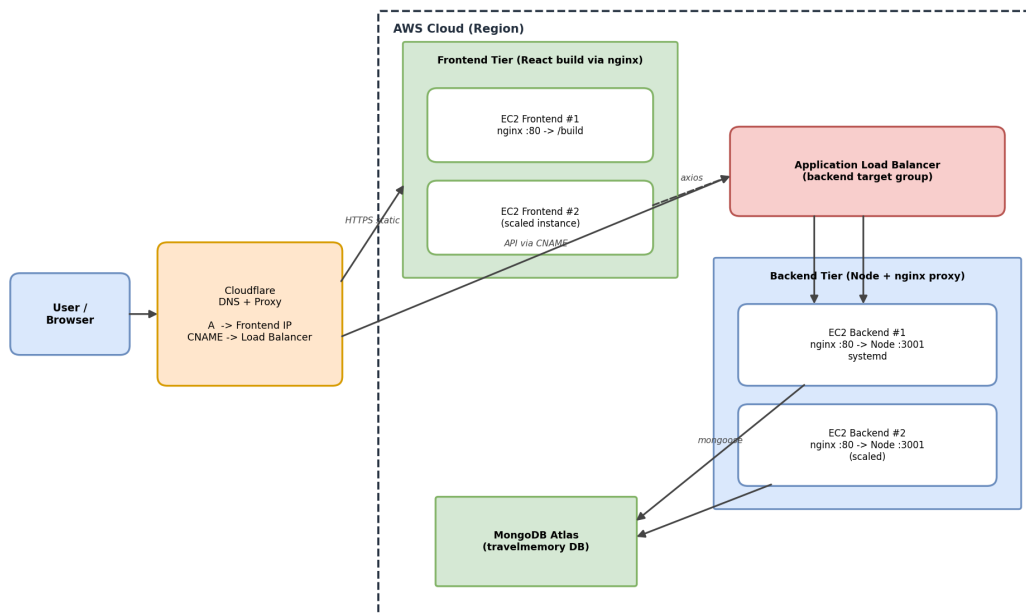
TravelMemory — EC2 Deployment Architecture

Figure 1 — Deployment architecture (draw.io)

3. Backend & Database Configuration

The repository was cloned to the instance and backend dependencies installed. MongoDB was installed directly on the EC2 instance, listening on 127.0.0.1:27017. The backend `conn.js` reads `MONGO_URI`, set in `backend/.env` along with the port:

```
cd /home/ec2-user && git clone https://github.com/UnpredictablePrashant/TravelMemory.git
cd TravelMemory/backend && npm install
# backend/.env:
MONGO_URI='mongodb://127.0.0.1:27017/travelmemory'
PORT=3001
```

A POC script (`test-connection.js`) verified connectivity before launching the app. The output confirms a successful connection and ping to the local MongoDB, and the backend registered as a `systemd` service shows active (running):

```

[ec2-user@ip-172-31-41-133 TravelMemory]$ ls
LICENSE README.md azure-pipelines.yml backend frontend
[ec2-user@ip-172-31-41-133 TravelMemory]$ cd backend
[ec2-user@ip-172-31-41-133 backend]$ ls
index.js controllers routes node_modules package-lock.json package.json routes
[ec2-user@ip-172-31-41-133 backend]$ cat conn.js
const mongoose = require('mongoose')
const URL = process.env.MONGO_URI

mongoose.connect(URL)
mongoose.Promise = global.Promise

const db = mongoose.connection
db.on('error', console.error.bind(console, 'DB ERROR: '))

module.exports = {db, mongoose}
[ec2-user@ip-172-31-41-133 backend]$ nano test-connection.js
[ec2-user@ip-172-31-41-133 backend]$ MONGO_URI="mongodb://127.0.0.1:27017/travelmemory" node test-connection.js
MongoDB connection POC
MUI in use mongodb://127.0.0.1:27017/travelmemory
SUCCESS: Connected to MongoDB.
  host : 127.0.0.1
  port : 27017
  database: travelmemory
  ping : OK (server responded)
  closed connection cleanly. POC passed.
[ec2-user@ip-172-31-41-133 backend]$ nano .env
[ec2-user@ip-172-31-41-133 backend]$ node index.js
Server started at http://localhost:3001
[ec2-user@ip-172-31-41-133 backend]$ nano .env
[ec2-user@ip-172-31-41-133 backend]$ node index.js
Server started at http://localhost:3001
[ec2-user@ip-172-31-41-133 backend]$ sudo nano /etc/systemd/system/travelmemory.service
[ec2-user@ip-172-31-41-133 backend]$ sudo systemctl daemon-reload
[ec2-user@ip-172-31-41-133 backend]$ sudo systemctl enable --now travelmemory
Created symlink /etc/systemd/system/multi-user.target.wants/travelmemory.service - /etc/systemd/system/travelmemory.service.
[ec2-user@ip-172-31-41-133 backend]$ sudo systemctl status travelmemory
● travelmemory.service - TravelMemory Node.js Backend
   Loaded: loaded (/etc/systemd/system/travelmemory.service; enabled; preset: disabled)
   Active: active (running) since Sun 2024-06-29 01:03:03 UTC; 34s ago
     Main PID: 39537 (node)
       Tasks: 3 (limit: 1024)
        Memory: 30.3M
          CPU: 48ms
    CGroup: /system.slice/travelmemory.service
            └─39537 /usr/bin/node index.js

Jun 28 01:03:03 ip-172-31-41-133.ap-south-1.compute.internal systemd[1]: Started travelmemory.service - TravelMemory Node.js Backend.
Jun 28 01:03:03 ip-172-31-41-133.ap-south-1.compute.internal node[39537]: Server started at http://localhost:3001
[ec2-user@ip-172-31-41-133 backend]$

```

Figure 2 — MongoDB POC passed (Connected, Ping OK) and backend systemd service active (running)

4. nginx Reverse Proxy & Security Group

nginx was configured as a reverse proxy so the backend (port 3001) is reachable on port 80. On Amazon Linux 2023 the config lives in `/etc/nginx/conf.d/`, and the SELinux boolean `httpd_can_network_connect` had to be enabled or nginx returns 502 when proxying to Node. The screenshot confirms the backend active, SELinux boolean set, nginx config test successful, nginx active, and nginx listening on `0.0.0.0:80`:

```

[ec2-user@ip-172-31-41-133 backend]$ sudo nano /etc/systemd/system/travelmemory.service
[ec2-user@ip-172-31-41-133 backend]$ sudo systemctl daemon-reload
[ec2-user@ip-172-31-41-133 backend]$ sudo systemctl enable --now travelmemory
Created symlink /etc/systemd/system/multi-user.target.wants/travelmemory.service - /etc/systemd/system/travelmemory.service.
[ec2-user@ip-172-31-41-133 backend]$ sudo systemctl status travelmemory
● travelmemory.service - TravelMemory Node.js Backend
   Loaded: loaded (/etc/systemd/system/travelmemory.service; enabled; preset: disabled)
   Active: active (running) since Sun 2024-06-29 01:03:03 UTC; 34s ago
     Main PID: 39537 (node)
       Tasks: 3 (limit: 1024)
        Memory: 30.3M
          CPU: 48ms
    CGroup: /system.slice/travelmemory.service
            └─39537 /usr/bin/node index.js

Jun 28 01:03:03 ip-172-31-41-133.ap-south-1.compute.internal systemd[1]: Started travelmemory.service - TravelMemory Node.js Backend.
Jun 28 01:03:03 ip-172-31-41-133.ap-south-1.compute.internal node[39537]: Server started at http://localhost:3001
[ec2-user@ip-172-31-41-133 backend]$ sudo nano /etc/nginx/conf.d/travelmemory-backend.conf
[ec2-user@ip-172-31-41-133 backend]$ sudo systemctl -f httpd_can_network_connect 1
[ec2-user@ip-172-31-41-133 backend]$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
[ec2-user@ip-172-31-41-133 backend]$ sudo systemctl enable --now nginx
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service - /usr/lib/systemd/system/nginx.service.
[ec2-user@ip-172-31-41-133 backend]$ sudo systemctl reload nginx
[ec2-user@ip-172-31-41-133 backend]$ sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Sun 2024-06-29 01:10:44 UTC; 5min ago
     Process: 39861 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
     Process: 39862 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
     Process: 39863 ExecStartPre=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
     Process: 39872 ExecReload=/usr/sbin/nginx -s reload (code=exited, status=0/SUCCESS)
     Main PID: 39864 (nginx)
       Tasks: 3 (limit: 1024)
        Memory: 1.9M
          CPU: 110ms
    CGroup: /system.slice/nginx.service
            └─39864 "nginx" master process /usr/sbin/nginx
              └─39873 "nginx" worker process
                └─39874 "nginx" worker process

Jun 28 01:10:44 ip-172-31-41-133.ap-south-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Jun 28 01:10:44 ip-172-31-41-133.ap-south-1.compute.internal nginx[39863]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Jun 28 01:10:44 ip-172-31-41-133.ap-south-1.compute.internal nginx[39862]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Jun 28 01:10:44 ip-172-31-41-133.ap-south-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
Jun 28 01:11:00 ip-172-31-41-133.ap-south-1.compute.internal systemd[1]: Reloading nginx.service - The nginx HTTP and reverse proxy server...
Jun 28 01:11:00 ip-172-31-41-133.ap-south-1.compute.internal systemd[1]: Reloaded nginx.service - The nginx HTTP and reverse proxy server.
[ec2-user@ip-172-31-41-133 backend]$ sudo ss -tlnp | grep 80
LISTEN 0      511          0.0.0.0:80          0.0.0.0:*
           users:((("nginx",pid=39874,fd=8),("nginx",pid=39873,fd=8),("nginx",pid=39864,fd=8)))
[ec2-user@ip-172-31-41-133 backend]$

```

Figure 3 — nginx configured, SELinux boolean set, and nginx listening on port 80

Security Group inbound rules were configured to allow SSH (22), HTTP (80), and HTTPS (443):

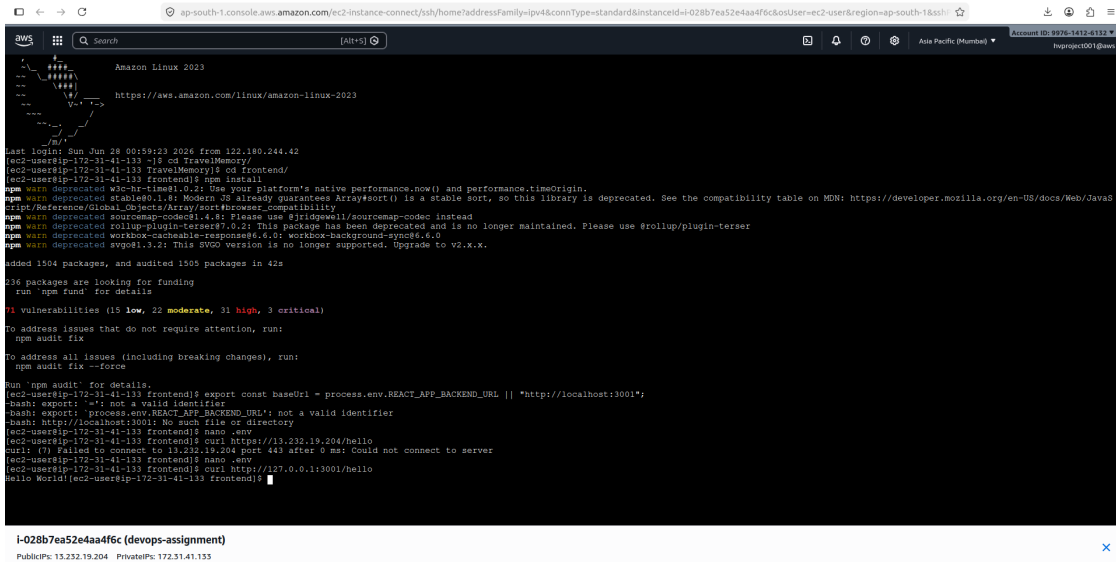


Figure 4 — Security Group inbound rules: SSH, HTTP, HTTPS (and port 3001 for debugging)

5. Frontend & Backend Connection

The frontend communicates through `baseUrl` in `frontend/src/url.js`, which reads `REACT_APP_BACKEND_URL`. A `.env` file was set to the custom domain, then the app was rebuilt (CRA bakes the variable in at build time) and published to the nginx web root with the SELinux file context restored:

```

cd /home/ec2-user/TravelMemory/frontend
echo 'REACT_APP_BACKEND_URL=https://trendlycart.in' > .env
npm run build
sudo cp -r build /var/www/travelmemory/
sudo restorecon -Rv /var/www/travelmemory # SELinux context (AL2023)
sudo nginx -t && sudo systemctl reload nginx

```



Figure 5 — frontend/.env set to `REACT_APP_BACKEND_URL=https://trendlycart.in`

[illegible]

Figure 6 — Build published to /var/www/travelmemory, SELinux context restored, nginx config test successful

With the build served at the root and API paths proxied to Node, the application loads and the backend /hello endpoint responds. Verified by IP before the domain was attached:

```

abhi@abhi: ~/Downloads$ curl http://13.232.19.204/hello
Hello World!
abhi@abhi: ~/Downloads$ curl http://13.232.19.204/
<!doctype html> <html lang="en"><head><meta charset="utf-8"/><link rel="icon" href="/favicon.ico"/><meta name="viewport" content="width=device-width,initial-scale=1"/><meta name="theme-color" content="#000000"/><meta name="description" content="web site created using create-react-app"/><link rel="apple-touch-icon" href="/logo192.png"/><link rel="manifest" href="/manifest.json"/><link href="https://cdn.jsdelivr.net/npm/bootstrap@3.0/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-pNdycU6t3KlIw0LlE03q7kO7mdZ6ldUOdK5J5Dq66zI6V8xQg7W8QGM932Zh3ewQPq0un7XGmHv6w3eG" crossorigin="anonymous"><script src="https://cdn.jsdelivr.net/npm/bootstrap@3.0/dist/js/bootstrap.bundle.min.js" integrity="sha384-w6lgg39R6crb3Bkw269x+JoqW6M3Bg9L3dSzGX4oIpwN93G8jvKLmgeWR7sRZ7kd" crossorigin="anonymous"></script><title>React App</title><script defer="defer" src="/static/js/main.2634be02.js"></script><link href="/static/css/main.073c9b0a.css" rel="stylesheet"></head><body><noscript>You need to enable JavaScript to run this app.</noscript><div id="root"></div></body></html>
abhi@abhi: ~/Downloads$

```

Figure 7 — Backend reachable via the instance IP: 13.232.19.204/hello returns Hello World!

Figure 8 — TravelMemory frontend served at 13.232.19.204 (Add Experience page)

6. Scaling the Application

To address load balancing through multiple instances, an Amazon Machine Image (AMI) was created from the fully configured instance, and a second instance (`new_instance_fromami`) was launched from it. The EC2 console shows both instances running in `ap-south-1`:

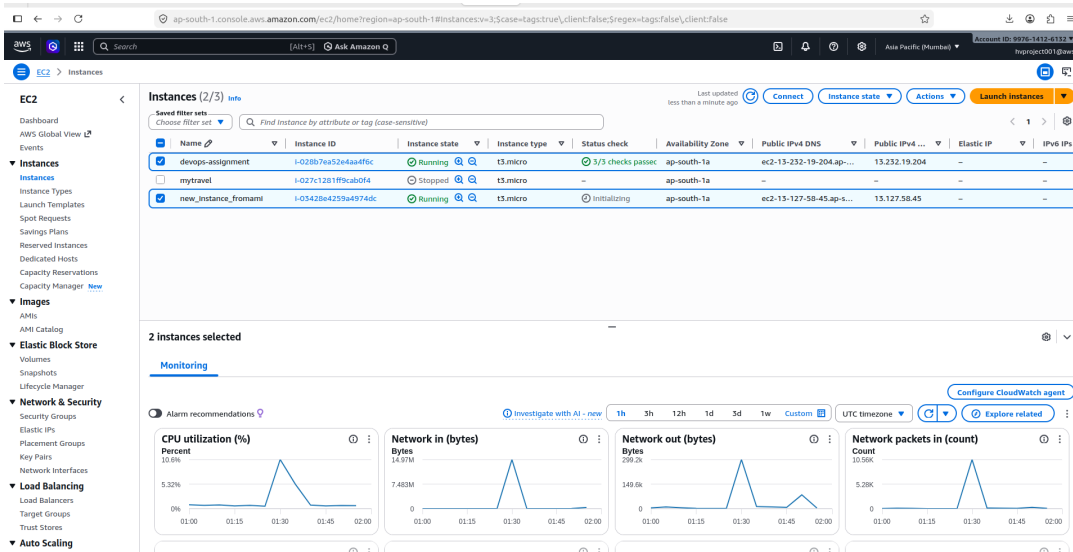


Figure 9 — Two running EC2 instances: `devops-assignment` (13.232.19.204) and `new_instance_fromami` (13.127.58.45)

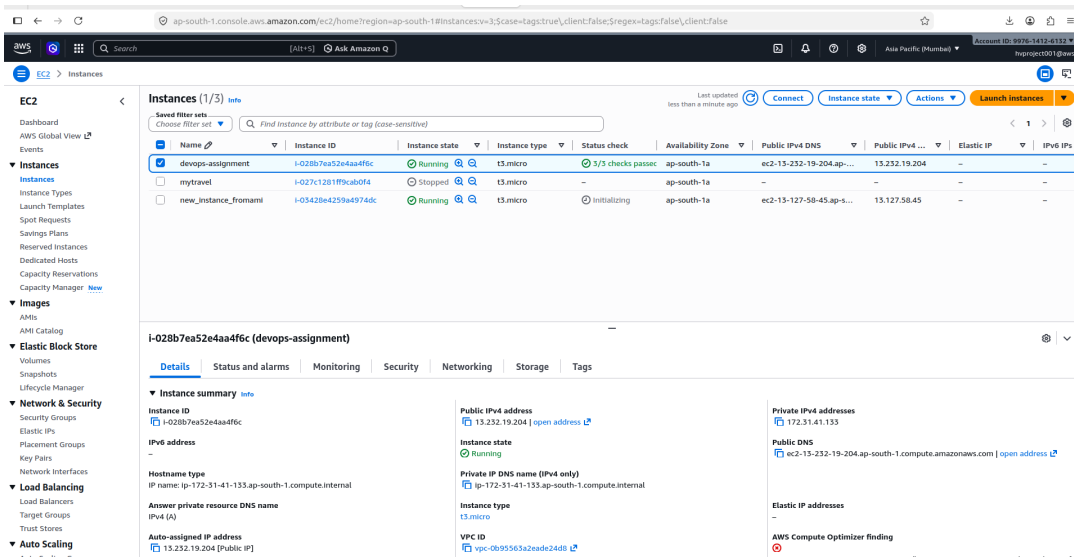


Figure 10 — Instance details: public IP 13.232.19.204, private IP 172.31.41.133

The second instance, cloned from the AMI, serves the same application independently — confirming the image is reusable for horizontal scaling behind a load balancer:



Figure 11 — Second instance (13.127.58.45) backend responding at /hello

```

[ec2-user@ip-172-31-35-129 ~]$ ls
TravelMemory
[ec2-user@ip-172-31-35-129 ~]$ cd TravelMemory/
[ec2-user@ip-172-31-35-129 TravelMemory]$ ls
LICENSE README.md azure-pipelines.yml backend frontend
[ec2-user@ip-172-31-35-129 TravelMemory]$ cd frontend
[ec2-user@ip-172-31-35-129 frontend]$ sudo nano /etc/nginx/conf.d/travelmemory-frontend.conf
[ec2-user@ip-172-31-35-129 frontend]$ ps aux | grep node
root      767  0.0  0.6   0   0  1*  02:02   0:00 [xfs:inodegc/num]
ec2-user  1724  0.0  7.2 651736 67628 7  Ssl  02:02   0:01 /usr/bin/node index.js
ec2-user  3260  0.0  0.2 222336 2156 pts/1  S+   02:36   0:00 grep --color=auto node
[ec2-user@ip-172-31-35-129 frontend]$

```

Figure 12 — Node process (*index.js*) running on the second instance

Scaling note: With multiple instances, a shared database (a dedicated MongoDB host on a private IP, or MongoDB Atlas) would be used so all backends read and write the same data. The local MongoDB suits the single-instance demonstration; the AMI proves the app tier scales horizontally.

7. Domain Setup with Cloudflare

The domain `trendlycart.in` was purchased from Hostinger. Cloudflare provided two nameservers, which were set as custom nameservers at Hostinger:

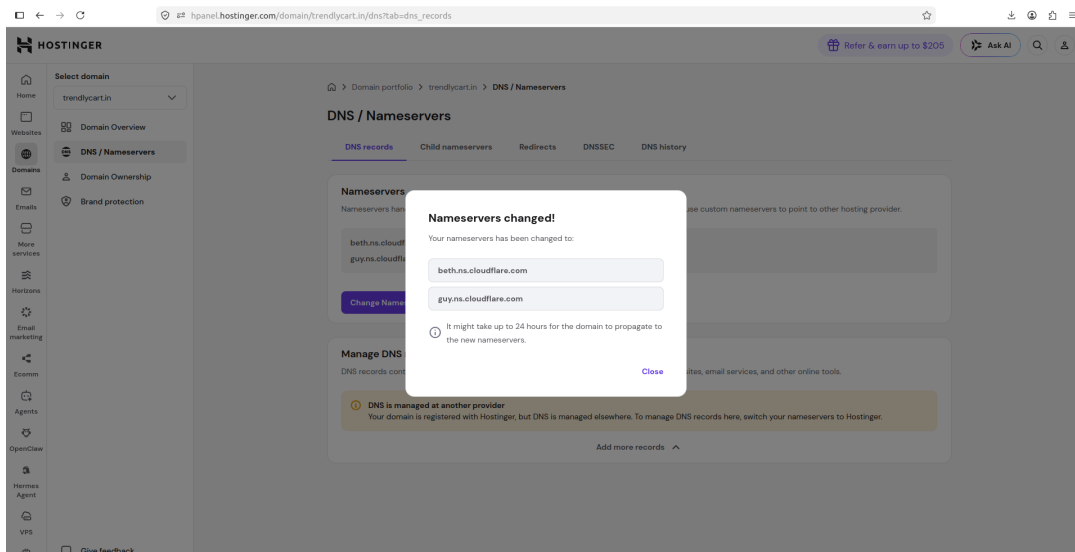


Figure 13 — Hostinger nameservers changed to Cloudflare (*beth.ns.cloudflare.com*, *guy.ns.cloudflare.com*)

7.1 DNS Records

In Cloudflare → DNS → Records, A records point the root and `www` to the EC2 instance IP. (Per the brief, in a full load-balancer design a CNAME would point to the ALB DNS name; with a single public instance the equivalent A record points directly to it.)

Type	Name	Value	Proxy	Purpose
------	------	-------	-------	---------

A	@	13.232.19.204	Proxied	Root → EC2 instance
A	www	13.232.19.204	Proxied	www → EC2 instance

After propagation, nslookup confirms the domain resolves to Cloudflare's proxy addresses rather than the origin IP, showing traffic flows through Cloudflare:

```
ssh@ec2:~/Downloads$ nslookup trendlycart.in
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
Name:   trendlycart.in
Address: 172.67.130.57
Name:   trendlycart.in
Address: 104.21.46.109
Name:   trendlycart.in
Address: 2606:4700:3032::6815:2e6d
Name:   trendlycart.in
Address: 2606:4700:3033::ac43:8a39
ssh@ec2:~/Downloads$
```

Figure 14 — nslookup trendlycart.in resolves to Cloudflare IPs (172.67.x / 104.21.x)

7.2 SSL/TLS Configuration

Cloudflare SSL/TLS mode was initially set to Full, producing a 521 error because Cloudflare attempts HTTPS to the origin (port 443) while the EC2 nginx serves HTTP on port 80. Switching the mode to Flexible resolved it — Cloudflare serves HTTPS at the edge and connects to the origin over HTTP. The application then loaded over HTTPS at the custom domain:



Figure 15 — Backend via custom domain: https://trendlycart.in/hello returns Hello World!

Travel Memory Add Experience

Trip Name
Add your Trip Name

Trip Date
dd/mm/yyyy

Name of Hotels
Add your Hotel Name

Trip Type
Select One

Total Cost
0

Places Visited
Delhi, Paris, London, etc.

Featured Trip?
☐ True
☒ False

Image Link
http://xyz.com/image.png

Short Description
Write Short Description

Experience
Write Complete Details about your experience in the trip.

Figure 16 — TravelMemory live over HTTPS at <https://trendlycart.in>

8. Verification Summary

Check	Evidence	Result
MongoDB connection (POC ping)	Figure 2	Passed
Backend systemd service active	Figures 2, 3	Passed
nginx listening on port 80	Figure 3	Passed
Security Group rules (80/443/22)	Figure 4	Passed
Frontend served + /hello via IP	Figures 7, 8	Passed
Second instance from AMI working	Figures 9–12	Passed
Nameservers on Cloudflare	Figure 13	Passed
Domain resolves via Cloudflare	Figure 14	Passed
Live over HTTPS at trendlycart.in	Figures 15, 16	Passed

9. Challenges & Resolutions

Issue	Cause	Resolution
MongooseError: uri undefined	MONGO_URI not loaded from .env	Set correct variable and local MongoDB URI
External curl to :80 timed out	Security Group missing inbound rule	Added HTTP (80) and HTTPS (443) rules
Cannot GET / at root	Backend nginx caught root path	Served React build at /, proxied API paths to Node
nginx 502 on proxy	SELinux blocking nginx	setsebool -P

	(AL2023)	httpd_can_network_connect 1
403 on static files	Wrong SELinux context on /var/www	restorecon -Rv /var/www/travelmemory
Cloudflare 521	SSL mode Full vs HTTP-only origin	Set Cloudflare SSL/TLS to Flexible

10. Deliverables & Submission

- A functional TravelMemory application on Amazon EC2, live over HTTPS at <https://trendlycart.in/hello>, <https://trendlycart.in/addexperience> via Cloudflare.
- This deployment documentation with step-by-step process and screenshots (Figures 1–16).
- A deployment architecture diagram created in draw.io (Figure 1).
- Horizontal scaling demonstrated via a second instance launched from an AMI.

— End of Document —